# How to Build Data Systems - Project Part 2

Juliane (**juhv**@itu.dk) & Mie (**miejo**@itu.dk)

November 10, 2026

## 1 Versioning

A table consists of two kinds of file. Partition files hold the rows; they are written once and never modified. The catalog, `catalog.json`, records which partition files currently constitute the table, together with their per-column min/max. A version is the catalog. The counter is a single field, `public int version`, at the top of `CatalogData`, beside the schema, the partition list and `maxPartitionNumber` — the highest partition number ever issued, never lowered, which guarantees that no two partition files share a name. Versions are per table, not global. `CREATE TABLE` writes version 0; every committed `COPY` or `DELETE` writes the next version over the same file. A `DELETE` matching no rows writes no new version.

A commit happens in two layers. On disk, `CatalogStore.write` serialises the new catalog to `catalog.json.tmp` and renames it over `catalog.json` with `ATOMIC_MOVE`. In memory we never mutate a catalog object already in use: we build a new `CatalogData` with the new partition list and `version + 1`, and the commit ends with a single assignment, `catalogs.put(tableName, next)`. Disk first and memory second, so a failed rename leaves the previous version in force and we never publish a version we did not persist. Validation, write and swap all run inside `synchronized (commitLocks.get(table))`, giving one writer per table in the commit section at a time.

The swap is atomic in both layers, for different reasons. On disk, bytes are only ever written into `.tmp`, and the rename replaces a single directory entry in one operation, so a reader always opens one whole catalog, old or new. In memory, `catalogs` is a `ConcurrentHashMap` and `put` is one atomic publication of an object built to completion beforehand, so no reader can observe a catalog under construction. Atomic is not the same as durable, however: we do not call `fsync`, so a power loss immediately after a commit can lose it.

Readers take no lock: `Planner.plan` snapshots the partition list at planning time, so a commit during a `SELECT` cannot change its result. No older versions are retained, and partition files that no catalog references are deleted when a `StorageEngine` opens and closes.

### 1.1 Worked example: version 1 to version 2 of a DELETE4

Four rows are loaded into a single partition of a table `trips_demo`, and `DELETE FROM`

`trips_demo WHERE city = 'Aarhus'` then removes one of them. Figure 1 shows the fields of `catalog.json` that differ across that commit. Appendix A shows details of the `catalog.json` and the records inserted into the table.

```
- "version"            : 1
- "maxPartitionNumber" : 0
+ "version"            : 2
+ "maxPartitionNumber" : 1

- "dataFile" : "partition-0.
    bin"
- "rowCount" : 4
+ "dataFile" : "partition-1.
    bin"
+ "rowCount" : 3

- "distance" : [ 12,   187
    ]
- "price"    : [ 23.5, 301.0
    ]
+ "distance" : [ 12,   95
    ]
+ "price"    : [ 23.5, 120.75
    ]
```

Figure 1: Excerpt of the catalog of `trips_demo` before and after `DELETE FROM trips_demo WHERE city = 'Aarhus'`, from a single-writer run. Every other field, the schema and the `city` range among them, is unchanged. Reproduce with `generate-catalog-evidence.sh`.

The version and `maxPartitionNumber` both rise by one, since a new partition file was handed a number. The partition was replaced rather than edited: `partition-0.bin` is unlinked, `partition-1.bin` holds the three surviving rows, and `rowCount` falls to 3. The statistics matter more. The deleted row held the maximum of both `distance` and `price`, and min/max are recomputed on the new file rather than carried over, so both maxima fall: pruning is strictly sharper after the delete, not merely correct. The `city` range is untouched, since `Aarhus` was neither its minimum nor its maximum. `partition-0.bin` is deleted when the engine closes.

## 2 Conflict detection

We adopt the optimistic concurrency approach, minimizing lock contention, and rebasing changes on commit. The first committer is the winner of a conflicting concurrency race. We do this by obtaining a lock on the table and checking whether the version has changed while we were doing work (i.e. comparing our base version to the catalog version on file).

Initially, we made an implementation that aborted in all cases of the version having been updated by other processes. The final implementation does rebasing for cases that manipulate disjoint sets of partitions, relaxing the strictness.

An interesting extension would be to to track row-level changes, since edits in disjoint rowsets could then be rebased successfully rather than aborted. This would require changes in the metadata for version-tracking beyond a list of partitions and code for handling reapplying row-changes based on the new partition files. A simpler extension would be to simply auto-retry the statement that failed.

### 2.1 Example Interleaving

An example that causes a Conflict Exception is the following interleaving of two DELETE statements, operating a delete on the same partition:

1. Thread A pins version 1
2. Thread A does work to delete city = "Copenhagen" from partitions 0, 1 & 2
3. Thread B pins version 1
4. Thread B does work to delete city = "Aarhus" from partition 0
5. Thread B commits version 2
6. Thread A wants to commit, but the version changed
7. Thread A attempts to rebase, but partition 0 is gone (1 and 2 are still there)
8. Thread A raises a ConflictException
9. Thread C reruns what Thread A failed on the base of version 2 and succeeds, committing version 3.

The full log of the evidence run is available in Appendix B. The interesting log-lines are particularly those seen in Figure 2, displaying the summary line of the completed commit of Thread B, the conflict and error of Thread A and the rerun success of Thread C.
This is a conflict because both DELETE statements manipulated the same partition. Since the versioning does not contain information about the exact manipulation, we cannot further clarify how to resolve the conflict and simply choose to abort. The winner is the one who commits first and the loser leaves behind the rewritten partitions which are left as orphans. A rerun of the loser will run from the new catalog table version and redo all steps creating new partition files. In our system, orphaned partitions are never overwritten and the partition numbers will increase for both committed and lost changes.

# 3 Reflections

**The storage layer:** Concurrent writers cost us coordination on the catalog update, and extra work when a commit conflicts. A `COPY` or `DELETE` builds its next catalog from the version it pinned and publishes it under the table's lock: one rename of `catalog.json`, then one map assignment. If another commit has landed first, the loser also has to rebase its partition list onto the winner's version before that publish. None of that surprised us. Two writers of the same catalog file have to agree on an order, and the order is by first committer.
What did surprise us was the necessity to extend the naming of partition files. A name has to stay unique for as long as any flow still references that file, including a writer that later loses the commit. We keep `maxPartitionNumber` in the catalog and never lower it, even if a partition is orphaned, and we hand numbers out atomically when requested, so one writer cannot overwrite a `partition-<n>.bin` that another flow wrote.

**ACID:** One statement is one transaction in our engine, and the four properties fare very differently.
*Atomicity we have.* A statement's effect is exactly one catalog version, published by a single rename followed by a single map assignment under the table's lock. Either every partition a `DELETE` rewrote becomes visible together, or none of it does.
*Consistency we have only in a weak sense.* We enforce the schema at bind time and when coercing values during `COPY`, and a commit preserves our structural invariants: every cata-

```
1  (Thread B) op=delete table=trips rowsDeleted=1 partitionsRewritten
   =1 partitionsCarried=3 version=2 durationMs=2
2  (Thread A) conflict table=trips base=1 current=2 type=
   removed_partitions_overlap
3  (Thread A) op=delete table=trips durationMs=4 outcome=FAILED error
   =ConflictException message=conflict table=trips base=1 current=2
   type=removed_partitions_overlap
4  (Thread C) op=delete table=trips rowsDeleted=3 partitionsRewritten
   =3 partitionsCarried=1 version=3 durationMs=1
```

Figure 2: Log line excerpts from a failing interleaving of DELETE statements. We denote Threads with letters corresponding to the interleaving description in Section 2.1. Line 1 shows the commit of Thread B succeeding on executing the DELETE, Line 2 shows the rebase discovering the conflict, Line 3 shows the raised Conflict Exception and Line 4 shows the successful rerun of Thread A's DELETE.

log entry points at a file that exists, and every partition's min/max describe the exact rows in the partition. We have no keys, no uniqueness and no referential integrity, so there is little for consistency to mean beyond "the catalog always describes a readable table".

*Isolation we approximate.* A `SELECT` pins its partition list at planning time and reads that version to completion, whatever commits meanwhile. At commit, an overlapping rewrite set aborts the loser, ruling out lost updates; disjoint sets rebase to an outcome still equivalent to a serial order, the loser running *before* the winner. The limit is scope: our lock is a Java monitor, so two processes on the same data directory share no lock.

*Durability we do not have.* We never call `fsync`, so a commit is visible but not guaranteed to survive a power loss. What we lose is bounded: the rename is atomic, so a crash leaves the previous version intact plus some orphaned partition files, which are cleaned on the next open. "Committed" in our engine means visible to every reader, not written through to disk.

**The retrofit:** A lot of the decisions we made in Part 1 helped us in our further development; our engine already swapped catalogs by writing a `.tmp` file and doing an atomic swap and holding a snapshot of the catalog in-memory as it was read. Also, keeping a catalog for each table ensured we could easily lock on per-table granularity in a straightforward fashion when committing. On of the main challenges / surprises was the modifications we needed due to the way we kept track of which partition numbers had been handed out already as previously described. Most modules stayed similar with minor changes and we just had to add code to handle concurrency and conflict detection / resolution for writers.

In our case, designing versioning from week 1 would probably not have yielded any different results or different workload, as our decisions already shaped a direction that fed well into the versioning we designed later on. We do recognize that the use of AI tools may have aided in this, as we sometimes arrived at better long-term-durable solutions due to

the AI models suggesting better ways to do things. For example, the use of `.tmp` files and atomic swaps for the `catalog.json` arrived in discussions with an AI agent about risks of inconsistency in a single-user session, but also addresses issues in the concurrent users setting.

**AI usage:** Our use of AI was pretty similar to Project 1. The agentic model writes the code based on clear requirements, exercise markdown and discussions with us. We often challenge decisions and give follow-up instructions for implementations where we are dissatisfied with coverage or approach. As described above, we see that using AI agents with follow-up discussions lead to better long-term solutions and less bugs and rewrites later on. We write the report ourselves and also wrote design decisions ourselves but subsequently discuss decisions and pros/cons with the AI.

# A Catalog Before & After a DELETE

This Appendix exemplifies what happens to the catalog file of a specific table 'trips' when running the following SQL script:

```
CREATE TABLE trips (city STRING, distance LONG, price DOUBLE);
COPY trips FROM 'trips.csv';
DELETE FROM trips WHERE city = 'Aarhus';
```

with `trips.csv` containing the following records:

```
Copenhagen,12,23.5
Aarhus,187,301.0
Odense,95,120.75
Aalborg,31,45.0
```

This corresponds to running the evidence-bash-script in the source code repository [1]. The outputs generated below were run on commit `284fe11`. Version 1 right after the initial copy uses partition number 0:

```
{
  "version" : 1,
  "maxPartitionNumber" : 0,
  "columns" : [ {
    "name" : "city",
    "type" : "STRING"
  }, {
    "name" : "distance",
    "type" : "LONG"
  }, {
    "name" : "price",
    "type" : "DOUBLE"
  } ],
  "maxRowsPerPartition" : 10000,
  "partitions" : [ {
    "dataFile" : "partition-0.bin",
    "rowCount" : 4,
    "stats" : {
      "city" : {
        "min" : "Aalborg",
        "max" : "Odense"
      },
      "distance" : {
        "min" : 12,
        "max" : 187
      },
      "price" : {
        "min" : 23.5,
        "max" : 301.0
```

[1] `https://github.com/julianefalsig/datasys-engine-TheQueryCrew/blob/main/experiments/project_2/generate-catalog-evidence.sh`

```
      }
    }
  } ]
}
```

Once the DELETE has been run, the version is increased, a partition is dropped and another partition is added:

```
{
  "version" : 2,
  "maxPartitionNumber" : 1,
  "columns" : [ {
    "name" : "city",
    "type" : "STRING"
  }, {
    "name" : "distance",
    "type" : "LONG"
  }, {
    "name" : "price",
    "type" : "DOUBLE"
  } ],
  "maxRowsPerPartition" : 10000,
  "partitions" : [ {
    "dataFile" : "partition-1.bin",
    "rowCount" : 3,
    "stats" : {
      "city" : {
        "min" : "Aalborg",
        "max" : "Odense"
      },
      "distance" : {
        "min" : 12,
        "max" : 95
      },
      "price" : {
        "min" : 23.5,
        "max" : 120.75
      }
    }
  } ]
}
```

# B Full Log Lines of failing DELETE interleaving

This section contains the full log lines after running the evidence-bash-script provided in the code base next to the source code [2]. The given log is from a run on commit `dfc9e44`.

```
2026-11-05 10:36:28.104,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=removeOrphans when=open removed=0
2026-11-05 10:36:28.138,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=createTable table=trips columns=3 version=0
2026-11-05 10:36:28.140,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=copyFile table=trips partition=0 column=
    city min=Aarhus max=Copenhagen
2026-11-05 10:36:28.140,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=copyFile table=trips partition=0 column=
    distance min=12 max=187
2026-11-05 10:36:28.140,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=copyFile table=trips partition=0 column=
    price min=23.5 max=301.0
2026-11-05 10:36:28.140,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=copyFile table=trips partition=1 column=
    city min=Copenhagen max=Odense
2026-11-05 10:36:28.140,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=copyFile table=trips partition=1 column=
    distance min=95 max=140
2026-11-05 10:36:28.141,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=copyFile table=trips partition=1 column=
    price min=120.75 max=210.0
2026-11-05 10:36:28.141,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=copyFile table=trips partition=2 column=
    city min=Aalborg max=Roskilde
2026-11-05 10:36:28.141,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=copyFile table=trips partition=2 column=
    distance min=31 max=210
2026-11-05 10:36:28.141,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=copyFile table=trips partition=2 column=
    price min=45.0 max=340.5
2026-11-05 10:36:28.141,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=copyFile table=trips partition=3 column=
    city min=Copenhagen max=Esbjerg
2026-11-05 10:36:28.141,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=copyFile table=trips partition=3 column=
    distance min=88 max=299
2026-11-05 10:36:28.141,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=copyFile table=trips partition=3 column=
    price min=99.99 max=450.25
2026-11-05 10:36:28.146,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=copyFile table=trips file=/var/folders/2q/9
```

[2] https://github.com/julianefalsig/datasys-engine-TheQueryCrew/blob/main/experiments/project_2/generate-conflict-evidence.sh

```
    y3zc43s0h5f_z2nr_1rmx0h0000gn/T/junit-12376374860663269219/trips.
    csv rows=8 partitions=4 version=1 durationMs=6
2026-11-05 10:36:28.148,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,
    DEBUG,StorageEngine,op=delete table=trips partition=4 column=city
    min=Aarhus max=Aarhus
2026-11-05 10:36:28.148,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,
    DEBUG,StorageEngine,op=delete table=trips partition=4 column=
    distance min=187 max=187
2026-11-05 10:36:28.148,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,
    DEBUG,StorageEngine,op=delete table=trips partition=4 column=price
     min=301.0 max=301.0
2026-11-05 10:36:28.148,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,
    DEBUG,StorageEngine,op=delete table=trips partition=0 decision=
    REWRITE rowsDeleted=1 rowsKept=1
2026-11-05 10:36:28.148,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,
    DEBUG,StorageEngine,op=delete table=trips partition=5 column=city
    min=Odense max=Odense
2026-11-05 10:36:28.148,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,
    DEBUG,StorageEngine,op=delete table=trips partition=5 column=
    distance min=95 max=95
2026-11-05 10:36:28.148,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,
    DEBUG,StorageEngine,op=delete table=trips partition=5 column=price
     min=120.75 max=120.75
2026-11-05 10:36:28.149,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,
    DEBUG,StorageEngine,op=delete table=trips partition=1 decision=
    REWRITE rowsDeleted=1 rowsKept=1
2026-11-05 10:36:28.149,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,
    DEBUG,StorageEngine,op=delete table=trips partition=2 decision=
    CARRY reason=noMatch
2026-11-05 10:36:28.149,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,
    DEBUG,StorageEngine,op=delete table=trips partition=6 column=city
    min=Esbjerg max=Esbjerg
2026-11-05 10:36:28.149,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,
    DEBUG,StorageEngine,op=delete table=trips partition=6 column=
    distance min=299 max=299
2026-11-05 10:36:28.149,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,
    DEBUG,StorageEngine,op=delete table=trips partition=6 column=price
     min=450.25 max=450.25
2026-11-05 10:36:28.149,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,
    DEBUG,StorageEngine,op=delete table=trips partition=3 decision=
    REWRITE rowsDeleted=1 rowsKept=1
2026-11-05 10:36:28.150,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=delete table=trips partition=7 column=city
    min=Copenhagen max=Copenhagen
2026-11-05 10:36:28.150,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,
    DEBUG,StorageEngine,op=delete table=trips partition=7 column=
    distance min=12 max=12
```

```
2026-11-05 10:36:28.150,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=7 column=price min=23.5 max=23.5
2026-11-05 10:36:28.150,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=0 decision=REWRITE rowsDeleted=1 rowsKept=1
2026-11-05 10:36:28.150,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=1 decision=CARRY reason=pruned
2026-11-05 10:36:28.150,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=2 decision=CARRY reason=noMatch
2026-11-05 10:36:28.150,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=3 decision=CARRY reason=pruned
2026-11-05 10:36:28.151,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips rowsDeleted=1 partitionsRewritten=1 partitionsCarried=3 version=2 durationMs=2
2026-11-05 10:36:28.151,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,DEBUG,StorageEngine,conflict table=trips base=1 current=2 type=removed_partitions_overlap
2026-11-05 10:36:28.151,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,38,ERROR,StorageEngine,op=delete table=trips durationMs=4 outcome=FAILED error=ConflictException message=conflict table=trips base=1 current=2 type=removed_partitions_overlap
2026-11-05 10:36:28.177,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,SqlParser,statements=1 durationMs=8
2026-11-05 10:36:28.178,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,StorageEngine,op=schema table=trips columns=3
2026-11-05 10:36:28.179,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,Planner,partition=0 decision=READ reason=noPredicate
2026-11-05 10:36:28.179,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,Planner,partition=1 decision=READ reason=noPredicate
2026-11-05 10:36:28.179,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,Planner,partition=2 decision=READ reason=noPredicate
2026-11-05 10:36:28.179,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,Planner,partition=3 decision=READ reason=noPredicate
2026-11-05 10:36:28.179,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,Planner,op=select table=trips partitionsTotal=4 partitionsRead=4 partitionsPruned=0 version=2
2026-11-05 10:36:28.179,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,ScanOperator,op=scan table=trips partitions=4 rowsOut=7
2026-11-05 10:36:28.179,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,Executor,select_complete rowsOut=7 parseMs=9 bindMs=1 planMs=0 executeMs=0 durationMs=2 version=2
2026-11-05 10:36:28.180,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=0 decision=REWRITE rowsDeleted=1 rowsKept=0
```

```
2026-11-05 10:36:28.180,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=8 column=city min=Odense max=Odense
2026-11-05 10:36:28.180,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=8 column=distance min=95 max=95
2026-11-05 10:36:28.180,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=8 column=price min=120.75 max=120.75
2026-11-05 10:36:28.180,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=1 decision=REWRITE rowsDeleted=1 rowsKept=1
2026-11-05 10:36:28.180,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=2 decision=CARRY reason=noMatch
2026-11-05 10:36:28.181,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=9 column=city min=Esbjerg max=Esbjerg
2026-11-05 10:36:28.181,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=9 column=distance min=299 max=299
2026-11-05 10:36:28.181,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=9 column=price min=450.25 max=450.25
2026-11-05 10:36:28.181,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips partition=3 decision=REWRITE rowsDeleted=1 rowsKept=1
2026-11-05 10:36:28.181,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,StorageEngine,op=delete table=trips rowsDeleted=3 partitionsRewritten=3 partitionsCarried=1 version=3 durationMs=1
2026-11-05 10:36:28.181,it-29497aa9-19de-4f06-82b2-6dffb89b302d,0,3,DEBUG,SqlParser,statements=1 durationMs=0
2026-11-05 10:36:28.181,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,StorageEngine,op=schema table=trips columns=3
2026-11-05 10:36:28.182,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,Planner,partition=0 decision=READ reason=noPredicate
2026-11-05 10:36:28.182,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,Planner,partition=1 decision=READ reason=noPredicate
2026-11-05 10:36:28.182,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,Planner,partition=2 decision=READ reason=noPredicate
2026-11-05 10:36:28.182,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,Planner,op=select table=trips partitionsTotal=3 partitionsRead=3 partitionsPruned=0 version=3
2026-11-05 10:36:28.182,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,ScanOperator,op=scan table=trips partitions=3 rowsOut=4
2026-11-05 10:36:28.182,it-29497aa9-19de-4f06-82b2-6dffb89b302d,1,3,DEBUG,Executor,select_complete rowsOut=4 parseMs=0 bindMs=0 planMs=0 executeMs=0 durationMs=0 version=3
```